

Lab 04: KNN Classification

Amogh Sahore

2547108

Machine Learning

Aim

To implement KNN classification on the breast cancer dataset and analyse model performance using train-test split, heuristic K selection, cross-validation, ROC-AUC, and classification metrics. Also, to compare classification metrics with regression metrics studied in Linear Regression.

Dataset

Breast Cancer Wisconsin Diagnostic dataset is used. The dataset has 569 records, 30 numerical features and one target column.

Target used here:

- 0 = Malignant
- 1 = Benign

Problem Statement

A healthcare analytics team needs a KNN model for early cancer detection. The model has to be tested using different splits, K values, cross-validation and classification metrics.

Task 1: Data Preparation

Load the dataset, check its structure, missing values and duplicates. Apply feature scaling because KNN is distance based.

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_curve, roc_auc_score
)
```

```
pd.set_option("display.max_columns", 40)
RANDOM_STATE = 42
```

```
In [2]: df = pd.read_csv("brca.csv")
df = df.rename(columns={df.columns[0]: "id"})

df.head()
```

```
Out[2]:
```

	id	x.radius_mean	x.texture_mean	x.perimeter_mean	x.area_mean	x.smoothness_mean	x.comp
0	1	13.540	14.36	87.46	566.3	0.09779	
1	2	13.080	15.71	85.63	520.0	0.10750	
2	3	9.504	12.44	60.34	273.9	0.10240	
3	4	13.030	18.42	82.61	523.8	0.08983	
4	5	8.196	16.84	51.71	201.9	0.08600	

```
In [3]: print("Shape:", df.shape)
print("Missing values:", df.isnull().sum().sum())
print("Duplicate rows:", df.duplicated().sum())
print("Target values:")
print(df["y"].value_counts())
```

```
Shape: (569, 32)
Missing values: 0
Duplicate rows: 0
Target values:
y
B    357
M    212
Name: count, dtype: int64
```

```
In [4]: df["target"] = df["y"].map({"M": 0, "B": 1})

X = df.drop(columns=["id", "y", "target"])
y = df["target"]

print("Feature shape:", X.shape)
print("Target shape:", y.shape)
```

```
Feature shape: (569, 30)
Target shape: (569,)
```

Inference: The data has no missing values. The target is converted into numeric form. Scaling is needed because KNN calculates distance, so large-value features can dominate small-value features.

Task 2: Train-Test Split Analysis

Use 80:20, 70:30 and 90:10 splits and compare performance.

```
In [5]: def make_knn(k):
        return Pipeline([
            ("scaler", StandardScaler()),
            ("knn", KNeighborsClassifier(n_neighbors=k))
```

```

    ])

def malignant_scores(y_true, y_pred):
    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision_malignant": precision_score(y_true, y_pred, pos_label=0),
        "recall_malignant": recall_score(y_true, y_pred, pos_label=0),
        "f1_malignant": f1_score(y_true, y_pred, pos_label=0)
    }

split_rows = []

for test_size in [0.20, 0.30, 0.10]:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, stratify=y, random_state=RANDOM_STATE
    )
    k = round(np.sqrt(len(X_train)))
    if k % 2 == 0:
        k += 1

    model = make_knn(k)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    row = {
        "split": f"{int((1-test_size)*100)}:{int(test_size*100)}",
        "train_size": len(X_train),
        "test_size": len(X_test),
        "k_used": k
    }
    row.update(malignant_scores(y_test, y_pred))
    split_rows.append(row)

split_results = pd.DataFrame(split_rows)
split_results

```

Out[5]:

	split	train_size	test_size	k_used	accuracy	precision_malignant	recall_malignant	f1_malignant
0	80:20	455	114	21	0.964912	1.0	0.904762	0.95000
1	70:30	398	171	21	0.947368	1.0	0.859375	0.92437
2	90:10	512	57	23	0.982456	1.0	0.952381	0.9756

Inference: The scores may change slightly when the split changes because the training and testing samples are different. A single split is useful, but it is not enough to fully judge the model.

Task 3: KNN Model with Heuristic K Selection

Use `K = sqrt(n)` as the first K value, then check nearby K values. Also compare distance metrics and plot decision boundaries.

```

In [23]: X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, stratify=y, random_state=RANDOM_STATE
    )

```

```

heuristic_k = round(np.sqrt(len(X_train)))
if heuristic_k % 2 == 0:
    heuristic_k += 1

print("Training samples:", len(X_train))
print("Heuristic K:", heuristic_k)

```

Training samples: 455

Heuristic K: 21

```

In [21]: k_values = list(range(max(1, heuristic_k - 5), heuristic_k + 6))
k_rows = []

for k in k_values:
    model = make_knn(k)
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    k_rows.append({
        "k": k,
        "accuracy": accuracy_score(y_test, y_pred),
        "recall_malignant": recall_score(y_test, y_pred, pos_label=0),
        "f1_malignant": f1_score(y_test, y_pred, pos_label=0)
    })

k_results = pd.DataFrame(k_rows)
k_results

```

Out[21]:

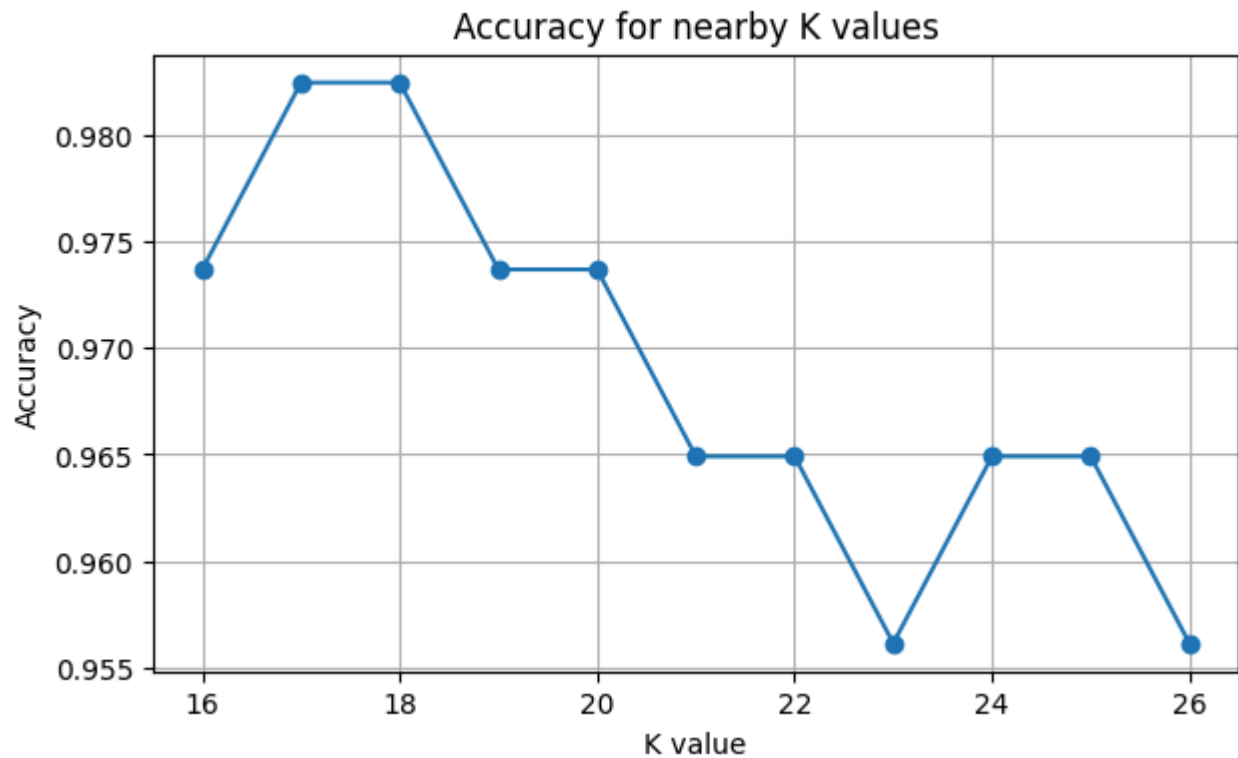
	k	accuracy	recall_malignant	f1_malignant
--	---	----------	------------------	--------------

0	16	0.973684	0.952381	0.963855
1	17	0.982456	0.952381	0.975610
2	18	0.982456	0.952381	0.975610
3	19	0.973684	0.928571	0.962963
4	20	0.973684	0.928571	0.962963
5	21	0.964912	0.904762	0.950000
6	22	0.964912	0.904762	0.950000
7	23	0.956140	0.880952	0.936709
8	24	0.964912	0.904762	0.950000
9	25	0.964912	0.904762	0.950000
10	26	0.956140	0.904762	0.938272

```

In [22]: plt.figure(figsize=(7, 4))
plt.plot(k_results["k"], k_results["accuracy"], marker="o")
plt.xlabel("K value")
plt.ylabel("Accuracy")
plt.title("Accuracy for nearby K values")
plt.grid(True)
plt.show()

```



```
In [16]: metric_rows = []

for metric in ["euclidean", "manhattan"]:
    model = Pipeline([
        ("scaler", StandardScaler()),
        ("knn", KNeighborsClassifier(n_neighbors=heuristic_k, metric=metric))
    ])
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    metric_rows.append({
        "metric": metric,
        "accuracy": accuracy_score(y_test, y_pred),
        "recall_malignant": recall_score(y_test, y_pred, pos_label=0),
        "f1_malignant": f1_score(y_test, y_pred, pos_label=0)
    })

pd.DataFrame(metric_rows)
```

```
Out[16]:
```

	metric	accuracy	recall_malignant	f1_malignant
0	euclidean	0.964912	0.904762	0.950000
1	manhattan	0.947368	0.904762	0.926829

```
In [17]: boundary_features = ["x.radius_mean", "x.texture_mean"]
X_two = X[boundary_features]

x_min, x_max = X_two.iloc[:, 0].min() - 1, X_two.iloc[:, 0].max() + 1
y_min, y_max = X_two.iloc[:, 1].min() - 1, X_two.iloc[:, 1].max() + 1
xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 250),
    np.linspace(y_min, y_max, 250)
)
grid = pd.DataFrame(np.c_[xx.ravel(), yy.ravel()], columns=boundary_features)
```

```

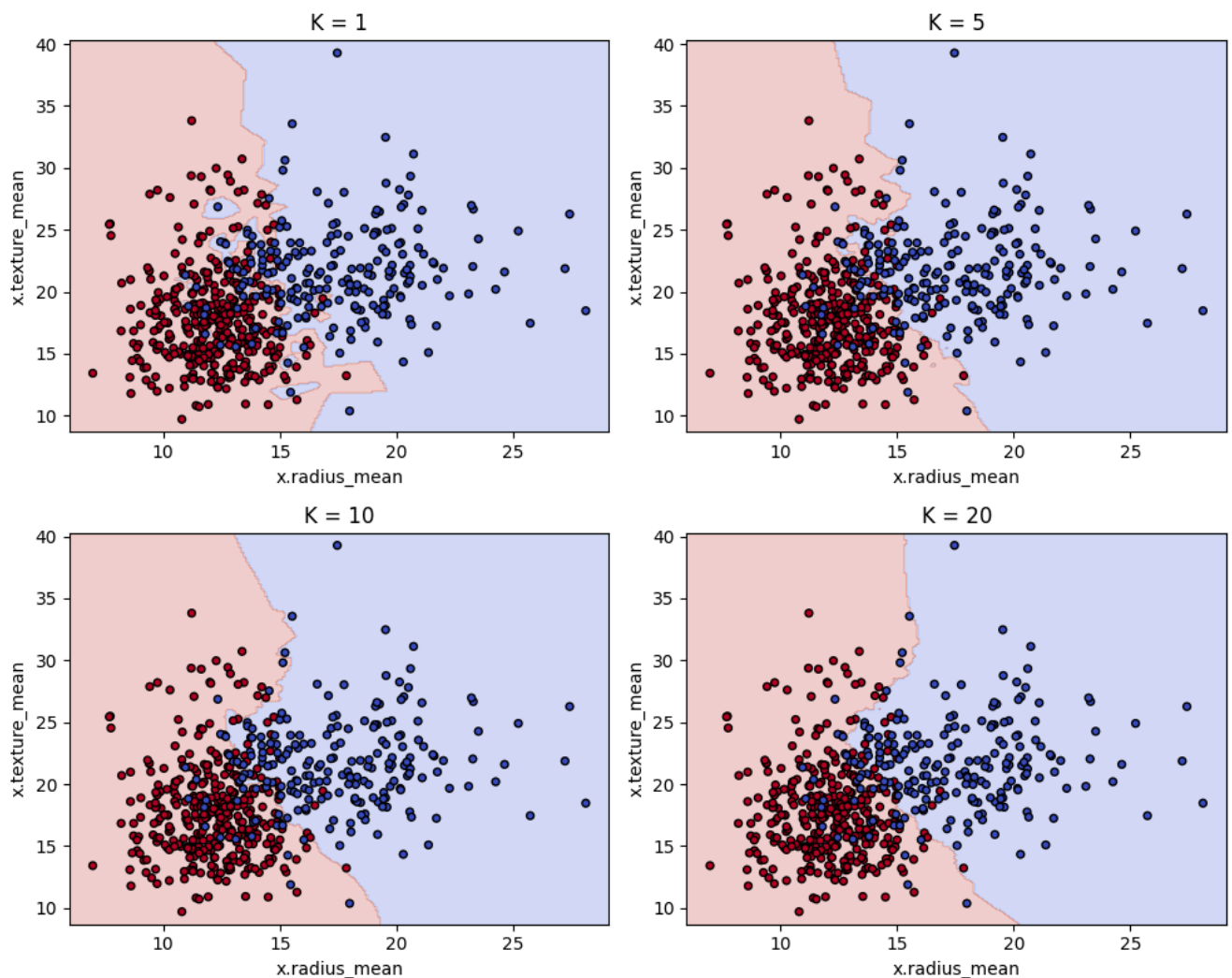
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

for ax, k in zip(axes.ravel(), [1, 5, 10, 20]):
    model = make_knn(k)
    model.fit(X_two, y)
    Z = model.predict(grid).reshape(xx.shape)

    ax.contourf(xx, yy, Z, alpha=0.25, cmap="coolwarm")
    ax.scatter(X_two.iloc[:, 0], X_two.iloc[:, 1], c=y, cmap="coolwarm", s=18, edgecolor="k")
    ax.set_title(f"K = {k}")
    ax.set_xlabel(boundary_features[0])
    ax.set_ylabel(boundary_features[1])

plt.tight_layout()
plt.show()

```



Inference: Smaller K gives a more irregular boundary and can overfit. Larger K gives a smoother boundary but very high K can miss local patterns.

Euclidean distance: Straight-line distance, useful when features are continuous and scaled.

Manhattan distance: Sum of absolute differences, useful when movement or difference is considered feature-wise.

Task 4: Cross Validation

Use K-Fold cross-validation and select K using average accuracy.

```
In [12]: cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
cv_rows = []

for k in range(1, 41, 2):
    model = make_knn(k)
    scores = cross_val_score(model, X_train, y_train, cv=cv, scoring="accuracy")
    cv_rows.append({
        "k": k,
        "mean_accuracy": scores.mean(),
        "std_accuracy": scores.std()
    })

cv_results = pd.DataFrame(cv_rows)
best_k = int(cv_results.loc[cv_results["mean_accuracy"].idxmax(), "k"])

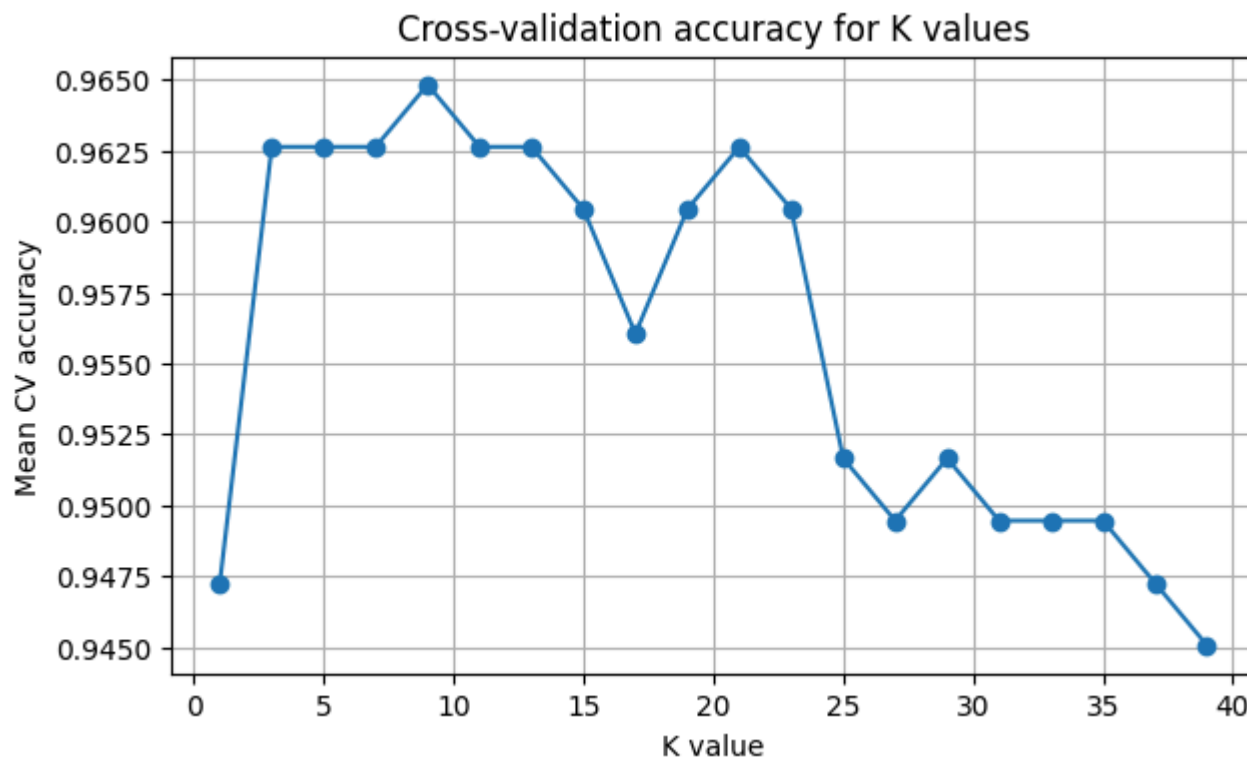
print("Best K from CV:", best_k)
cv_results.sort_values("mean_accuracy", ascending=False).head(10)
```

Best K from CV: 9

```
Out[12]:
```

	k	mean_accuracy	std_accuracy
4	9	0.964835	0.014579
5	11	0.962637	0.017855
10	21	0.962637	0.014906
1	3	0.962637	0.008791
2	5	0.962637	0.011207
3	7	0.962637	0.020382
6	13	0.962637	0.020382
11	23	0.960440	0.014906
9	19	0.960440	0.016447
7	15	0.960440	0.016447

```
In [13]: plt.figure(figsize=(7, 4))
plt.plot(cv_results["k"], cv_results["mean_accuracy"], marker="o")
plt.xlabel("K value")
plt.ylabel("Mean CV accuracy")
plt.title("Cross-validation accuracy for K values")
plt.grid(True)
plt.show()
```



Inference: Cross-validation is better than relying on one split because every fold gets a chance to be tested. The final K is selected from the best mean CV accuracy.

Task 5: Classification Evaluation

Evaluate the final model using accuracy, precision, recall, F1 score, confusion matrix, ROC curve and AUC.

```
In [14]: final_model = make_knn(best_k)
final_model.fit(X_train, y_train)

y_pred = final_model.predict(X_test)
malignant_index = list(final_model.named_steps["knn"].classes_).index(0)
y_score_malignant = final_model.predict_proba(X_test)[ :, malignant_index]

print("Best K:", best_k)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision for malignant:", precision_score(y_test, y_pred, pos_label=0))
print("Recall for malignant:", recall_score(y_test, y_pred, pos_label=0))
print("F1 score for malignant:", f1_score(y_test, y_pred, pos_label=0))
print("ROC-AUC for malignant:", roc_auc_score(1 - y_test, y_score_malignant))
```

```
Best K: 9
Accuracy: 0.9736842105263158
Precision for malignant: 1.0
Recall for malignant: 0.9285714285714286
F1 score for malignant: 0.9629629629629629
ROC-AUC for malignant: 0.9943783068783068
```

```
In [16]: cm = confusion_matrix(y_test, y_pred, labels=[0, 1])
cm_df = pd.DataFrame(cm, index=["Actual M", "Actual B"], columns=["Pred M", "Pred B"])
cm_df
```


Out[16]:

	Pred M	Pred B
Actual M	39	3
Actual B	0	72

In [17]: `print(classification_report(y_test, y_pred, target_names=["Malignant", "Benign"]))`

```

              precision    recall  f1-score   support

 Malignant       1.00        0.93     0.96         42
    Benign       0.96        1.00     0.98         72

 accuracy              0.97         114
 macro avg           0.98     0.96     0.97         114
 weighted avg        0.97     0.97     0.97         114

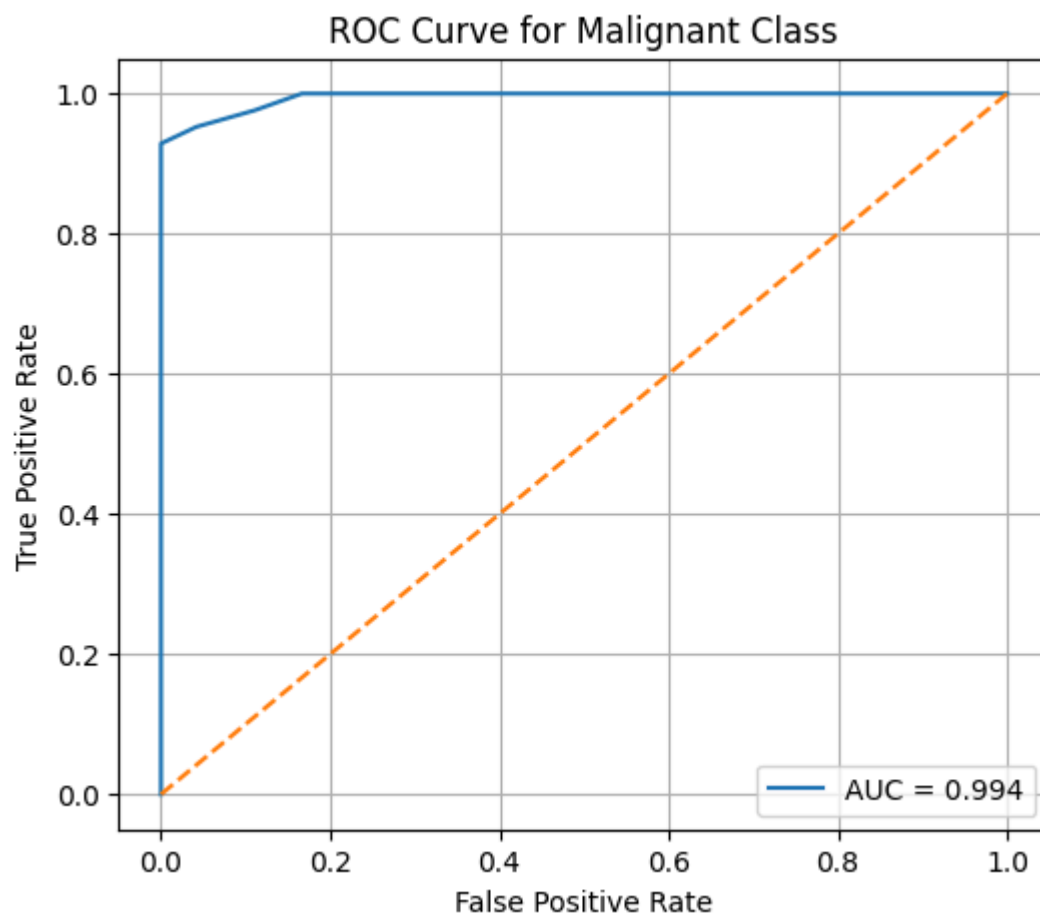
```

In [15]: `fpr, tpr, thresholds = roc_curve(1 - y_test, y_score_malignant)
auc_value = roc_auc_score(1 - y_test, y_score_malignant)`

```

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, label=f"AUC = {auc_value:.3f}")
plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve for Malignant Class")
plt.legend()
plt.grid(True)
plt.show()

```



Inference: In cancer prediction, recall for the malignant class is very important because missing a malignant case is more serious than wrongly flagging a benign case.

Task 6: Comparison with Regression Metrics

Regression metric	Classification metric	Difference
R ² score	Accuracy	R ² explains fit for continuous values. Accuracy counts correct classes.
RMSE	F1 score	RMSE measures numeric error size. F1 balances precision and recall.
MAE	Confusion matrix	MAE gives average error. Confusion matrix shows correct and wrong class decisions.

Regression metrics measure how far predicted values are from actual continuous values. Classification metrics check whether the predicted class is correct or not.

Accuracy alone is not enough in medical diagnosis because the model can still miss important malignant cases. Recall and ROC-AUC are more useful because they show how well the model identifies cancer cases and separates both classes.

Task 7: Analytical Questions

1. Why is KNN called a lazy learning algorithm?

KNN is called lazy because it does not build a proper model during training. It stores the data and does the main work during prediction.

2. Why is feature scaling required in KNN?

Scaling is required because KNN uses distance. Without scaling, features with large values affect the distance more.

3. Explain heuristic K selection using $\sqrt{n}$ rule.

A simple starting value for K is the square root of the number of training samples. Usually an odd value is preferred to reduce ties.

4. Why is cross-validation more reliable than a single train-test split?

Cross-validation tests the model on different folds, so the result does not depend only on one random split.

5. How does K affect bias-variance trade-off?

Small K has low bias but high variance. Large K has more bias but lower variance.

6. Why is recall more important than accuracy in cancer prediction?

Recall is important because it shows how many actual cancer cases were detected. Missing cancer cases is risky.

7. What is the limitation of very large K values?

Very large K can make the model too general and it may ignore local patterns in the data.

Conclusion

KNN was applied on the breast cancer dataset after scaling the features. The heuristic K value was used as a starting point and nearby K values were compared. Cross-validation was used to choose the final K more reliably. The final model was evaluated using accuracy, precision, recall, F1 score, confusion matrix and ROC-AUC.

For this problem, recall and ROC-AUC are more important than only accuracy because the dataset is related to medical diagnosis. Compared to regression metrics, classification metrics focus on correct class decisions instead of prediction error magnitude.